



Fakultät Informatik

From CAD Software to Slicer

Workflow Optimization using Print Hints in 3MF Files for 3D Printing

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Luise Hartdegen

Matrikelnummer 364 2168

Erstgutachter: Prof. Dr. Ulrich von Zadow

Zweitgutachter: Prof. Dr. Timo Götzelmann

© 2025

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

//TODO shorter motivation, more focus on realized solution; keep it general → don't mention 3MF, OrcaSlicer, Fusion

With the rising popularity and easier access to 3D printers, more and more people are interested in designing and printing their own models. There are however still a few steps involved that require domain specific knowledge, oftentimes creating a discrepancy between the wanted outcome and reality. Preparing a 3D model for printing requires the user to apply material presets and printing parameters after importing the model to a slicer software. Doing this once is a logical and necessary step, but when creating a 3D model from scratch or changing one, it might take several iterations until the result matches the desired outcome. Thus the user has to repeatedly apply the aforementioned settings and therefore they have to remember them or write them down which creates friction in the workflow. Not only does this increase cognitive workload, leading to a higher frequency of errors and thus produces waste, it also often leads to frustration in beginner or casual makers. The solution presented in this paper are so-called print hints added in the 3D modeling software itself, in the proof of concept realized using Fusion appearances. Upon opening a 3MF file in OrcaSlicer, the appearances are mapped to either material presets or modifiers. By giving the user the option to map manually while also saving and loading previous mappings, the solution provides an automation of the criticized workflow while keeping the freedom to easily change an entire group of objects in one simple step.

Kurzdarstellung

//TODO translation

Contents

1. Introduction	1
2. State of the Art Technologies and Workflow	3
2.1. State of the Art Workflow from Fusion to OrcaSlicer	4
2.1.1. Exporting as STL	5
2.1.2. Exporting as OBJ	6
2.1.3. Exporting as 3MF	6
2.1.4. Applying Materials	6
2.1.5. Applying Settings with and without Presets	7
2.2. The 3MF File Format	7
3. Problem	10
3.1. Negative Effects of Cognitive Workload	10
3.2. Sharing Models Online	11
3.3. Fabrication Documentation	12
4. Method	13
4.1. Writing print hints to 3MF file in Fusion	13
4.2. Analysis of OrcaSlicer's 3MF File Processing	13
4.3. Implementation Design	14
5. Results	16
6. Conclusion	17
6.1. Outlook	17
6.2. Summary	17
A. Supplemental Information	18
List of Figures	21
List of Tables	22
List of Listings	23
Bibliography	24

Chapter 1.

Introduction

//TODO revise

- start anew, ignoring the abstract
- focus on problem
- use workflow from literature as basis
- don't introduce figure so far ahead

It is one of the clearest paradoxes of modern work that computer-based systems designed to reduce task complexity and cognitive workload actually often impose even greater demands and stresses on the very individuals they are supposed to be helping.[1, p. 222]

When designing a model for 3D printing, the process frequently follows a certain set of iterative steps. After the ideation phase, the design engineer first creates the model using CAD software, imports the file (e.g. STL, 3MF) to a slicer software, sets the necessary parameters and finally starts printing. They evaluate the result for design flaws or areas of improvement, leading to model revisions and thereby reinitiating the workflow as pictured in figure 2.2. Throughout the iterations, some printing parameters will be tweaked but overall you want to keep them consistent to have a reliable comparison. Despite automation and optimization having been a prevalent topic in the software developer community for years, setting the parameters is generally a manual step. Hence, the design engineer has to remember their settings or save them some other way, possibly by writing them down meticulously or taking screenshots.

With access to printers in public institutions becoming easier and the growing popularity of 3D printing not only within the *maker* community but in casual users as well, HCI needs to focus on streamlining error-prone workflows like the one described above[2]. “[C]urrent tools that focus on supporting only one aspect of the workflow (*e.g.* , only modelling or only printing) may be less useful for casual makers and there is opportunity

in HCI research to innovate on design tools that take into account the interdependencies within the 3D printing workflow”[2] //TODO check if still up-to-date / relevant

This thesis focuses on the communication between CAD software and slicer, with a particular focus on proving the feasibility of ‘print hints’. //TODO introduce print hints in more detail

The goal is to streamline the iterative design workflow by reducing the need to repeatedly configure the same printing parameters in each iteration, therefore lowering cognitive workload for design engineers. The current prototype contains two core components: firstly, an improved export function within the CAD software *Autodesk Fusion*, e.g. to mark certain objects as modifiers; and second, automatic handling of materials and these print hints within the open source slicer software called *OrcaSlicer*.

basic (HCI) aspects in which I want to improve the workflow (possibly to be referenced in the following texts) → what does it solve

- sharing files with slicer but not printer specific settings
- how it reduces cognitive workload in repetitive steps

Chapter 2.

State of the Art Technologies and Workflow

- general workflow without specific software
- difference in representation of 3d models between digitally targeted software (Blender), manufacturing targeted (CAD) and slicers
-

“A standard 3D printing workflow, for example, requires disparate pieces of software, which then leave behind only a fragmented record of the fabrication process. CAD models are stored and shared as static geometry files. Separately, operators can tweak relevant settings in CAM software called a slicer, which can then be saved as a slicing profile.”[its not the shape]

“CAM-based design tools have emerged where toolpaths are specified directly. This approach is contrasted with CAD-based tools which are based on geometric models [23]. CAM-based tools naturally shift focus to the machine and its settings. Prior work generates custom toolpaths using traditional CAD environments [20 , 21], code [25 , 54 , 62], hand-drawn sketches [23], and domain-specific abstractions [5].”[its not the shape]

A short explanation of both softwares involved will provide context for the following examination of the current workflow and how it can be improved in more detail. The first software, Autodesk Fusion, is used for “design, engineering, electronics, and manufacturing [...] all-in-one”[3]. It provides functionalities for the “entire product development process [...] with integrated 3D CAD, CAM, CAE, and PCB”[3], meaning it can be used for designing parts to manufacture in either additive or subtractive processes. Mainly relevant to the process at hand is the CAD (Computer Aided Design) functionality. Contrary to 3D modelling software like Blender intended for purely digital products, CAD is based on precise measurements, taking into account not only the surface but also the body’s volume. It provides visualization tools for examining and analyzing designs, as well as simulation capabilities, for example, to assess how different components interact or respond to various mechanical stresses.

Slicer software on the other hand imports a 3D model and “slices [it] into layers for the printer [...] allow[ing] users to select print settings, such as layer height and print speed”[2] It therefore translates the model into step-by-step commands for a specific printer. There are many different slicers available online nowadays, either general ones or ones specific to printer manufacturers. Since “getting all the settings right for a particular 3D printer can be fiddly, a lot of manufacturers modify one of these open source programs and create a proprietary program just for their machine.”[4, p. 58] *OrcaSlicer* is one the open source slicers, originally forked from *Bambu Studio*, which itself was forked from *PrusaSlicer*, a descendant of *Slic3r*. “Slic3r is licensed under the GNU Affero General Public License, version 3”[5], which means that every software using parts of it has to be licensed under the same license. Since these slicers are all intricately linked, oftentimes new features are being introduced in one of them and then pulled over into the others as well.

OrcaSlicer always handles files as projects, either by interpreting 3MF files as a project in itself or by opening a file within a new project. That way the user has the option to save project specific settings or presets for further use or just export the generated g-code for printing and discard said settings.

2.1. State of the Art Workflow from Fusion to OrcaSlicer

When analyzing the process from idea to print, several main steps can be identified as shown in 2.2. “[Y]ou need to create or acquire a 3D computer-aided design (CAD) model, slice this model into layers and generate the commands needed to physically create the model, and, finally, physically create the print.”[4, p. 52] Using a simple box with a snap-fit closure and two different filament colors as an example, the following analysis will focus on the steps from exporting a model to printing, highlighting in particular how the workflow differs depending on the file format used in regards to three points.

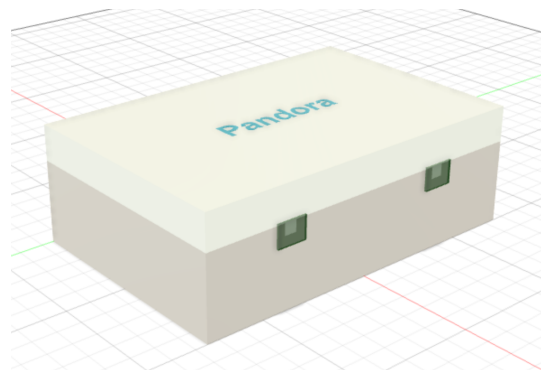


Figure 2.1.: 3D model of simple box

1. A model consisting of multiple parts, some of which are not positioned on the surface they need to be printed on (e.g. the lid)
2. Using multiple materials (e.g. printing the text in a different color)

3. Using model parts as modifiers (e.g. the translucent green boxes as modifiers to print more wall loops in the closure's area for higher stability)

Exporting the model from Fusion is possible via two dialogs: the general export and the 3D print specific one. The two dialogs each offer a different set of file formats due to their different purposes. Since this analysis deals with 3D printing in particular, only the export dialog specific to 3D printing will be considered, which offers to export as STL, 3MF and OBJ format. For all three the user can decide whether to generate one file containing all selected components and bodies, or one file per body.

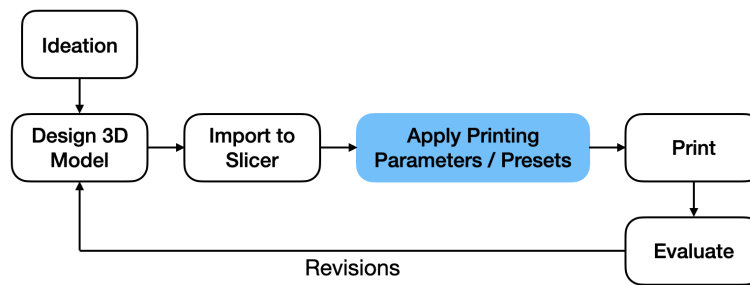


Figure 2.2.: Iterative workflow in 3D printing highlighting the step to be optimized

2.1.1. Exporting as STL

Exporting the model as an STL file immediately shows one important distinction. “STL native files describe only the surface geometry of a three dimensional object without any representation of color, texture or other common CAD model attributes,”[6] meaning that even if a user applies materials in Fusion, that information won’t be exported, and the user has to apply all materials anew in OrcaSlicer.

Furthermore there will always be one body in the resulting file, even if the model is comprised of multiple bodies within Fusion. Therefore the user has to choose whether to combine all bodies into one or exporting each body as a separate file. The first option results in one extra step to split the model into separate objects within OrcaSlicer to be able to print the text in a different color or to use the green boxes as modifiers. Alternatively exporting each body as a separate file might loose their spatial relations to each other while keeping the bodies separate. OrcaSlicer does however ask the user whether the files should be loaded as a single object with multiple parts. The advantage over loading each file as an object is the preservation of the spatial relation and thus parts, like the modifiers in the example box, can be placed at any height while a single object is always placed with at least one surface flush with the printing plate.

2.1.2. Exporting as OBJ

The OBJ file format “is the preferred 3D printer file format for multicolor printing”.[6] Consequently, OrcaSlicer shows a particular dialog (fig. A) when opening an OBJ file, asking the user how they want to map the file’s colors to current or new filament presets. This makes it easy for users to align filament colors (i.e. the color the filament is represented as in OrcaSlicer, not necessarily the color of the filament itself) to colors applied in Fusion. Just like STL files though, the model is generated as one body per file resulting in the same dialogs and extra steps.

2.1.3. Exporting as 3MF

Unlike the other two formats, 3MF not only preserves separate bodies even when exported as a single file making it more user-friendly, it also includes “information about the model’s color, material, and textures.”[6] Initially when importing a 3mf file containing multiple objects, OrcaSlicer asks the user whether the file should be loaded as a single object with multiple parts instead of considering them as multiple objects similar to the dialog when importing multiple STL or OBJ files at once.

While Fusion does export materials as RGBA colors in one hexadecimal color code per body, OrcaSlicer does not offer the user to map these to available or new presets so far. Consequently users have to reapply materials manually which has been criticized by users, e.g. in *OrcaSlicer issue #2438*¹.

2.1.4. Applying Materials

“For extrusion-based 3D printing, material properties such as rheology² or melting temperature are as relevant as machine configurations such as print speed, nozzle geometry, and heating profiles.” [It’s Not the Shape, It’s the Settings]

Generally, materials can be set using system or user defined presets per object. Material settings entail, amongst others, the filament itself, nozzle temperature, general printing speed and cooling settings. OrcaSlicer actively asks the user, whether they want to

¹Single Object Multiple parts with Multiple Colours: [...] I have made a number of 3D models with multiple parts in single 3mf file. I mostly use Fusion 360 for models and during modeling I assign colours to each part. But when I upload this 3mf file to the slicer, It assigns the primary filament colour to all the parts. I was wondering if the slicer would be able to distinguish the parts with different colours and assign either the original colour to the parts or even random colours(which can be changed later). For example if my 3mf file has 5 colour parts, the slicer should be to detect them. Then it should find which colour has the maximum spread by area and assign the filament 1 to it. The second largest should be assigned the second filament and so on till the last.[...] <https://github.com/SoftFever/OrcaSlicer/discussions/2438>

²[//TODO](#) explain

save any changes to material parameters they have made, which makes it easy to save either parameters specific to one's printer and filament or even specific to one project while simultaneously keeping default material presets untouched. When importing a new file or project, OrcaSlicer does apply the first available user preset filament by default. More detailed analysis even showed, that a multicolor file will be imported with different filaments assigned consecutively but the user doesn't have any options to map colors to filaments the way it works with OBJ files. Furthermore, the last color per colorgroup (more details in the following chapter 2.2) is assigned to each object which is why even complex models with different colors appear as having one single material in OrcaSlicer.

2.1.5. Applying Settings with and without Presets

Settings like the amount of wall loops, infill percentage and pattern, and printing speed for single layers might need to be customized. It is possible to save these as presets as well, however only globally, i.e. they can only be applied to the entire project, not to specific objects within. As criticized in the *OrcaSlicer Issue #7769*³, particularly for projects containing multiple objects, using presets in this case is not a valid option.

2.2. The 3MF File Format

While each of the three file formats mentioned have their own advantages and disadvantages, preliminary analysis indicated that the 3MF format represents the most suitable solution for the problem at hand. 3MF has been developed by the 3MF Consortium, founded by Microsoft, Autodesk, HP, 3D Systems, Prusa Research and several other key players in the field, to create a 3D printing file format that addresses the shortcomings of previous standards (STL, OBJ, AMF). In particular the possibility for extensions and the support of colors as well as several different 3D model representations were a main factor, making it ideal for state-of-the-art additive manufacturing.[7]

“A 3MF file is a ZIP archive that contains a complex file structure, defining a 3D model in Extensible Markup Language (XML) file”[7] usually called `3dmodel.model`, as seen in 2.3. Optionally, there can be multiple 3D model files, one of which needs to be identified in the

```
example.3mf
├── _rels/
├── [Content_Types].xml/
├── 3D/
│   ├── 3dmodel.model
│   └── Metadata/
│       └── thumbnail.png
```

³Copy/paste per-object settings: When planning/building large projects in-slicer it often becomes an extremely tedious task to click all the objects you want a specific set of settings to apply to. The larger the assembly/project you are planning, the easier it is to miss a body and make a mistake. If you "assemble" models together you cannot edit multiple sub-objects across assemblies.
<https://github.com/SoftFever/OrcaSlicer/issues/7769>

StartPart relationship as the root model. The 3D model part is comprised of two sections per specification: “a set of resource definitions that include objects and materials, as well as a set of specific items to actually build”.[8]

Taking a closer look at the composition of said XML file as it is exported from Fusion, see A.1, these sections can be found in lines 4 ff. (the **resources** tag), enclosing the **m:colorgroup** and **object** tags, and lines 41 ff. containing the **build** tag. While materials can be represented in a number of different resource tags (e.g. basematerials, m:texture2d, m:compositematerials...), Fusion seems to solely use the **m:colorgroup** tag. Unfortunately there aren’t a lot of information available as to the exact way Fusion exports 3MF, hence the following paragraph is partly based on explorative observations.

The **colorgroup** contains two or more colors as an RGBA hexadecimal color code each, as mentioned before. Per the additional Materials and Properties Extension specification[9], colors are specifically expressed in sRGB colorspace in accordance with the World Wide Web Consortium. Each group is identifiable by its **id** which is referenced in each object as **pid** while the colors themselves implicitly get a 0-based index. Despite the colors’ representation as an RGBA hexadecimal color code, thus including an 8-bit channel for opacity, the standard specifies that color properties *should not* reference translucent display properties. If required, a multi-properties group must be used instead. Empirical observations indicate compliance with this requirement, as the opacity value is consistently set to FF, i.e. a fully opaque color.

Since it is possible in Fusion to apply appearances and physical materials independently of one another, one might think one color represents the appearance and the other the physical material. Observations, however, do not support this theory since even when changing both, only the first color within one colorgroup changes while the second one stays a light grey (“#A0A0A0FF”).

The following **object** tags each contain attributes (body id, type, color id) as well as one triangular geometry mesh representation of an object volume. The latter is represented by vertices (x-, y-, z-coordinates in the unit specified within the model’s attributes) and triangles referencing three vertices each oriented with normals pointing towards the exterior of their object.[8] Meshes of different objects may intersect (as seen in the above mentioned example’s case of wall loop modifiers) or self-intersect, in which case it must be specified which areas are inside or respectively outside the volume.

A so-called PrintTicket can be attached to each 3D model part detailing user intent and device configuration information for printers. This might be useful for the intended

purpose of print hints but has not been further pursued as an idea due to the way, the file is being handled in *OrcaSlicer*.

In regards to adding print hints directly to the model's xml structure, it shall be noted that in accordance with the specification "XML content MUST NOT contain elements or attributes drawn from the "xml" or "xsi" namespaces unless they are explicitly defined in the XSD schema or by other means in the specification."[\[8\]](#)

Chapter 3.

Problem

As shown in the previous chapters 2.1.4 and 2.1.5, there are a lot of parameters to be repeatedly applied before a 3D model can be printed. So, why would this be a problem? There are in fact multiple points to consider regarding the necessity of repetitive manual steps in a common digital workflow especially in educational and casual makers' contexts.

“Current computer-aided design and manufacturing (CAD-CAM) software which assume a well-understood fabrication process are mismatched to a material-driven trial-and-error process. This creates friction when exploring—copying files, loading settings—but also produces downstream effects on how designs are stored, shared, and replicated. Slight variation in material composition and machine state affect physical outcomes”[its not the shape]

“We identify two important limitations of current fabrication software in material-driven practices: the ability to quickly explore a material’s design space, and the ability to document the insights gained through this exploration.”[its not the shape]

“tuning settings allows practitioners to experiment with surface texture and aesthetic properties, control material properties like stiffness, and optimize print settings”[same]

3.1. Negative Effects of Cognitive Workload

Setting the parameters once might be an engaging part of the entire process of getting to the final product. Especially in an educational context, the users do learn a lot about the possibilities the slicer and printer offer, the entire fabrication process and how to approach a project on their own. Doing the exact same steps repeatedly however, does not teach the users anything, while still requiring a certain amount of cognitive workload.

What does this even mean? As Kosch *et al.* summarized in [10], there are various definitions used in the field of human computer interaction (HCI). One of them differentiates

between *intrinsic* and *extraneous load*. The former encompasses the “inherent complexity of a task” and therefore “depends on the users individual cognitive resources and proficiency”, i.e. their skills. In other words, how difficult and extensive is a task to begin with and how well equipped is the person in dealing with the complexity. On the other hand, the latter describes cognitive resources needed due to the “task representation that can be perceived” by the users. For instance, how does the design of a user interface either reduce or increase friction within a given workflow?

Cognitive workload being a part of the workflow isn’t a negative aspect in itself. To a certain degree it is even necessary to foster task engagement which increases the learning effects and the feel of accomplishment. In the process at hand both types of cognitive workload can be identified. When initially setting the printing parameters, the task will be inherently complex; more or less so, depending on the complexity of the project itself. Each repetition however does turn into extraneous load, since having done the steps once before, the task requires focus without any added value of engagement, thus creating additional friction by way of design. One might argue that for beginners as well as casual users the step can nevertheless be overwhelming within the larger context of several new and complex tasks while for advanced makers it is simply tedious and time consuming.

Due to the task being only one part of a multifaceted list of steps, this kind of friction claims resources that won’t be free to solve other parts like setting up the printer. One might argue that people should just focus on one task at a time, plan ahead and not try to multitask, however with the added time constraint oftentimes present in particular in educational environments but also when using community equipment (e.g. in makerspaces), it becomes a necessary evil.

→ increasing amount of errors?

“In general, usability is an essential HCI concept and is concerned with making systems easy to learn, easy to use, and with minimal error frequency and severity.” [Sustainable Design]

TBC

3.2. Sharing Models Online

In their study of newcomers to 3D printing, Hudson *et al.* worked out a few key problems impacting the whole user experience, one of them being the problem of settings causing print failures.[2] While this can’t be entirely avoided through the solution presented in the following chapter, it can be reduced. Since creating a 3D model does present

an obstacle, people turn “to premade 3D models available through services such as *Thingiverse.com*”.[2]

However even when using such pre-made models, which oftentimes come with elaborate instructions regarding the printing parameters, users were successful only with the help of print center operators. Especially when 3D printers are provided by community institutions like libraries though, the staff might not be trained in the use of these printers as well as in makerspaces. This results in failed prints, frustrating the user but also creating material waste.

“With personal fabrication, however, the responsibility of waste generation is shifting directly to individual makers. As making becomes more accessible, the amount of material waste generated at the grassroots level is increasing.” [Make Making Sustainable]

“the responsibility of waste generation is shifting directly to individual makers”[dito]

“Almost all the makers discussed how their projects require design iterations and prototyping. These processes naturally generate a number of intermediate prototypes, which, as explained in Section 4.1.2, serve as a natural source of waste. Thus, optimizing design iterations and using fewer prototypes, ideally without compromising design goals, will result in reduced material usage, fewer physical artifacts, and more sustainable actions.”[dito]

TBC

“personal fabrication introduces a distinct challenge that was not present during the democratization of personal computing: the environmental impact of material waste produced by makers”[make making sustainable]

3.3. Fabrication Documentation

“Documenting fabrication workflows is a difficult and time-consuming activity. Practitioners often have to choose between making progress on their project and creating documentation”[its not the shape]

“most readers of documentation on the website Instructables are interested in customization, not rote recreation”[same]

“Currently when sharing 3D designs online, settings are optionally and manually disclosed alongside models.”[same]

Chapter 4.

Method

The task of setting printing parameters is by nature a complex one, due to the many different printers, materials and project-specific needs out there. To recap briefly, the intention is not to reduce that complexity in and of itself, but to reduce the need to remember and repeatedly apply those settings. Accordingly from the very start, the idea was to mark bodies with so-called ‘print hints’ in a 3D model in a way that enables the slicer to apply the user’s parameters automatically when importing a new model.

option 1: writing directly to the 3mf model_settings.config → complex

option 2: using Fusion’s native options to apply appearances and mapping those to filaments, printing parameters and modifier types using a custom mapping file and a dialog similar to the filament mapping when importing an .obj file

4.1. Writing print hints to 3MF file in Fusion

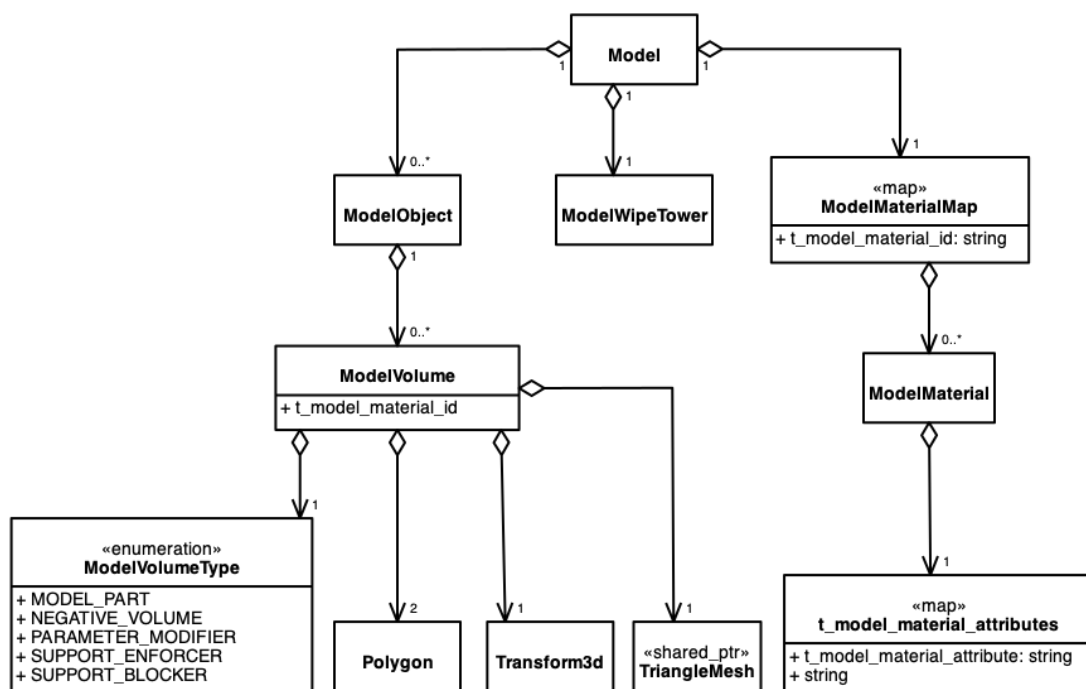
as a proof of concept → using Fusion’s native appearances (as opposed to physical materials) to mark differentiate bodies with diverging settings

these are already being saved and the big advantage is that users can create a personal library of appearances for all there models to reuse the appearance-to-parameter mapping

4.2. Analysis of OrcaSlicer’s 3MF File Processing

While chapter 2 put a focus on the state-of-the-art workflow from the user’s perspective, coming up with a way to solve the problem required a deeper analysis of the way OrcaSlicer handles 3mf files since this provided some clues as well as limitations regarding possible approaches.

- it does read in all the xml lines from the *3dmodel.model* file including the materials but it doesn't write them to the model
- the colors are being read in but due to the way Fusion saves colors in 3mf files (as explained previously), OrcaSlicer iterates over all colors per colorgroup and overwrites the 'correct' (first) color with the default grey which is usually the last color per colorgroup
- this was the easier part and first step to fix
- it does however map the colors automatically to extruders and assigns the user's saved filaments consecutively



4.3. Implementation Design

different possible approaches to solve the problem at hand and why did I do it the chosen way?

As mentioned before, as a proof of concept using specific materials to represent the possible modifiers was the way to go. The idea was to save user presets locally containing a map of “materials”, i.e. the RGBA hexadecimal color codes, to chosen modifier presets or actual filaments respectively. When opening a new 3MF project, the user would be prompted to either confirm the material to modifier / filament mappings or select options

for newly detected materials. This reduces the need to apply settings upon each import while simultaneously giving the user the option to easily change mappings.

- software architecture (diagram?)
- input, output
- choices that had to be made

Chapter 5.

Results

- what does the optimised workflow look like?
- why is the solution good?
 - how are the results analysed / validated?
 - openly available → advantages of that
 - but: it is a proof of concept / prototype and thus will most likely not be the end of the story

Chapter 6.

Conclusion

6.1. Outlook

- communication with OrcaSlicer's owner softfever?
- option of code being pulled into other slicers
- what is left to do?

6.2. Summary

Appendix A.

Supplemental Information

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <model xmlns="http://schemas.microsoft.com/3dmanufacturing/core
  /2015/02" unit="millimeter" xml:lang="en-US" ...>
3   <metadata name="Title" type="QName" preserve="1">Pandoras
    Box</metadata>
4   <resources>
5     <m:colorgroup id="2">
6       <m:color color="#B4B3B3FF"/>
7       <m:color color="#A0A0A0FF"/>
8     </m:colorgroup>
9     <m:colorgroup id="4">
10      <m:color color="#449648FF"/>
11      <m:color color="#A0A0A0FF"/>
12    </m:colorgroup>
13    ...
14    <object id="1" name="Body1" type="model" p:UUID="88b92134
      -b51b-4129-a501-3cb7a6b35a19" pid="2" pindex="0">
15      <mesh>
16        <vertices>
17          <vertex x="99.000000" y="-34.800000" z="
            32.500000" />
18          <vertex x="99.000000" y="-34.800000" z="
            35.000000" />
19          ...
20        </vertices>
21        <triangles>
22          <triangle v1="0" v2="1" v3="3" />
23          <triangle v1="3" v2="1" v3="2" />
24          ...
25        </triangles>
```

```

26         </mesh>
27     </object>
28     <object id="3" name="Body10" type="model" p:UUID="83
        b1ffc1-fe3a-41ff-b2d8-709e401b58c0" pid="4" pindex="0"
        >
29         <mesh>
30             <vertices>
31                 <vertex x="97.000000" y="44.000000" z="26.000000
                    " />
32                 ...
33             </vertices>
34             <triangles>
35                 <triangle v1="0" v2="1" v3="3" />
36                 ...
37             </triangles>
38         </mesh>
39     </object>
40 </resources>
41 <build p:UUID="9062348e-a030-4f92-86b2-786a03cd804d">
42     <item objectid="1" p:UUID="df0ef4a8-b05c-41c3-a041-
        f6ca9d41a68c" />
43     <item objectid="3" p:UUID="2dd63bd5-069d-405e-9b05-0
        fd5f5a71f72" />
44     ...
45 </build>
46 </model>

```

Listing A.1: Abbreviated Example of 3MF's 3dmodel.model File

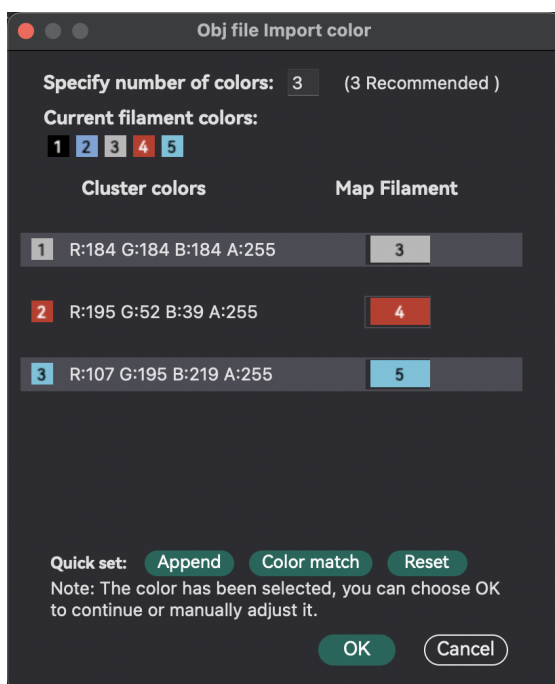


Figure A.1.: Dialog when opening an OBJ file in OrcaSlicer to map colors to filament presets cite software? & check formatting

List of Figures

2.1. 3D model of simple box	4
2.2. Iterative workflow in 3D printing highlighting the step to be optimized	5
2.3. Example structure of 3MF file	7
A.1. Dialog when opening an OBJ file in OrcaSlicer to map colors to filament presets cite software? & check formatting	20

List of Tables

List of Listings

A.1. Abbreviated Example of 3MF's 3dmodel.model File	18
--	----

Bibliography

- [1] J. A. Jacko, *Human Computer Interaction Handbook: Fundamentals, Evolving Technologies, and Emerging Applications, Third Edition*. Milton, UNITED KINGDOM: Taylor & Francis Group, 2012. [Online]. Available: <http://ebookcentral.proquest.com/lib/thnuernberg/detail.action?docID=911990>
- [2] N. Hudson, C. Alcock, and P. K. Chilana, “Understanding Newcomers to 3D Printing: Motivations, Workflows, and Barriers of Casual Makers,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 384–396. [Online]. Available: <https://dl.acm.org/doi/10.1145/2858036.2858266>
- [3] Autodesk Inc., “Autodesk Fusion | 3D CAD, CAM, CAE, & PCB Cloud-Based Software | Autodesk.” [Online]. Available: <https://www.autodesk.com/products/fusion-360/overview>
- [4] J. Horvath and R. Cameron, *Mastering 3D Printing in the classroom, library, and lab*. New York: Apress, 2018.
- [5] SoftFever, RF47, ianalexis, and Arcodude, “OrcaSlicer/doc.” [Online]. Available: <https://github.com/SoftFever/OrcaSlicer/tree/main/doc>
- [6] C. Iancu, “ABOUT 3D PRINTING FILE FORMATS,” *Annals of the „Constantin Brancusi” University of Targu Jiu*, no. 2, pp. 135–138, 2018.
- [7] J. Rossel, V. Mladenov, and J. Somorovsky, “Security Analysis of the 3MF Data Format,” in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. Hong Kong China: ACM, Oct. 2023, pp. 179–194. [Online]. Available: <https://dl.acm.org/doi/10.1145/3607199.3607216>
- [8] 3MF Consortium, “3MF Core Specification & Reference Guide,” International Organization for Standardization, Standard, Feb. 2025. [Online]. Available: <https://3mf.io/spec/core-v1-3-0/>
- [9] —, “Materials Extension,” International Organization for Standardization, Standard, Feb. 2025. [Online]. Available: <https://3mf.io/spec/materials-v1-2-1/>

- [10] T. Kosch, J. Karolus, J. Zagermann, H. Reiterer, A. Schmidt, and P. W. Woźniak, “A Survey on Measuring Cognitive Workload in Human-Computer Interaction,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–39, Dec. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3582272>